Fenwick Trees & Sparse Tables

Lighter tools for special cases: invertible sums and static idempotent queries

Learning goals

- Implement a **Fenwick tree** (Binary Indexed Tree) in ~10 lines
- Understand `i & -i` and which ranges BIT cells cover
- Use BITs for **range update / point query** and **range update / range query**
- Count **inversions** and find the **k-th element** with a BIT
- Build a **sparse table** for O(1) static RMQ
- Know when to pick BIT vs sparse table vs segment tree

The toolbox so far

Need	Tool
Static sums	prefix sums (Week 7)
Point update + any associative query	segment tree (Week 23)
Point update + **prefix sums**	**Fenwick tree** — today
Static **min/max/gcd**	**sparse table** — today
Range update + range query	lazy segment tree (Week 25)

Fenwick tree: the idea

Cell `t[i]` stores the sum of `a(i − lowbit(i), i]`, where `lowbit(i) = i & -i`.

```
i      :  1  2  3  4  5  6  7  8
lowbit :  1  2  1  4  1  2  1  8
covers : [1][1,2][3][1,4][5][5,6][7][1,8]
```

- 1-indexed! (`lowbit(0) = 0` would loop forever)
- `i & -i` isolates the lowest set bit (two's complement)

Prefix sum: strip low bits

```
long long prefix(int i) {              // a[1] + ... + a[i]
    long long s = 0;
    for (; i > 0; i -= i & -i) s += t[i];
    return s;
}
```

prefix(13): 13 = 1101_2 → t[13] covers (12,13], t[12] covers (8,12], t[8] covers (0,8]

At most log2 n steps — one per set bit.

Point update: add low bits

```
void add(int i, long long d) {       // a[i] += d
    for (; i <= n; i += i & -i) t[i] += d;
}
```

add(5): visit 5 → 6 → 8 → 16 → … — exactly the cells whose range contains 5.

Range sum `[l, r]` = `prefix(r) − prefix(l − 1)` (needs an **inverse** operation!)

Why it works (picture)



Query walks **left** through disjoint blocks; update walks **up** through every block covering i.

Build in O(n)

```
for (int i = 1; i <= n; i++) {
    t[i] += a[i];
    int j = i + (i & -i);
    if (j <= n) t[j] += t[i];
}
```

(Or just call `add` n times: O(n log n), usually fine.)

Application: inversion count

Pairs `i < j` with `a[i] > a[j]`:

- Compress values to ranks 1..m
- Scan left to right; for each `a[j]`: inversions += (# earlier elements) − (# earlier with rank ≤ rank(a[j]))
- Then `add(rank, 1)`

O(n log n). `2 4 1 3 5` → 3 inversions. Answer up to $n^2/2$ → `long long`.

Range update, point query

Keep a BIT over the **difference array** `d[i] = a[i] − a[i−1]`:

```
void rangeAdd(int l, int r, long long x) { add(l, x); add(r + 1, -x); }
long long pointQuery(int i) { return prefix(i); }
```

CSES 1651 *Range Update Queries*; path queries on a flattened tree (Week 22).

Range update, range query

Two BITs `B1`, `B2`. Then `prefix(i) = sum(B1, i) · i − sum(B2, i)`:

```
void rangeAdd(int l, int r, long long x) {
    upd(B1, l, x);           upd(B1, r + 1, -x);
    upd(B2, l, x * (l - 1)); upd(B2, r + 1, -x * r);
}
long long prefix(int i) { return qry(B1, i) * i - qry(B2, i); }
```

Derivation: `a[k] = Σ d[j]`, so `Σ_{k≤i} a[k] = Σ d[j]·(i − j + 1)` — split into `i·Σd[j]` and `Σd[j]·(j−1)`.

k-th element: binary lifting on the BIT

Smallest `p` with `prefix(p) ≥ k` (all values ≥ 0):

```
int kth(int k) {
    int pos = 0;
    for (int step = 1 << LOG; step > 0; step >>= 1)
        if (pos + step <= n && t[pos + step] < k) {
            pos += step;
            k -= t[pos];
        }
    return pos + 1;
}
```

`t[pos + step]` covers exactly `(pos, pos + step]` → O(log n), no binary search.

2D Fenwick tree

```
void add(int x, int y, long long d) {
    for (int i = x; i <= n; i += i & -i)
        for (int j = y; j <= m; j += j & -j) t[i][j] += d;
}
```

- Query rectangle by inclusion–exclusion of 4 prefixes
- O(log n · log m) per operation
- CSES 1739 *Forest Queries II*

Fenwick limitations

- Range query needs an **inverse**: sums, xor ✔; **min/max ✘** (cannot "subtract" a min)
- Prefix-min BIT works only if values only **decrease**
- For general range min/max with updates → segment tree

Sparse table: static RMQ

No updates. Precompute minima of all **power-of-two** windows:

```
sp[0][i] = a[i]
sp[k][i] = min(sp[k-1][i], sp[k-1][i + 2^(k-1)])     // window [i, i + 2^k)
```

n log n values, built in O(n log n).

O(1) query: overlapping windows

For `[l, r]`, let `k = ⌊log2(r − l + 1)⌋`:

```
min(a[l..r]) = min(sp[k][l], sp[k][r − 2^k + 1])

 l                              r
 [=========2^k=========]
           [=========2^k=========]
```

The two windows overlap — fine for **idempotent** operations: `min(x, x) = x`.

Sparse table code

```
lg[1] = 0;
for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
sp[0] = a;
for (int k = 1; k <= lg[n]; k++)
    for (int i = 0; i + (1 << k) <= n; i++)
        sp[k][i] = min(sp[k-1][i], sp[k-1][i + (1 << (k-1))]);

int query(int l, int r) {                    // inclusive
    int k = lg[r - l + 1];
    return min(sp[k][l], sp[k][r - (1 << k) + 1]);
}
```

Idempotent vs not

Operation	Idempotent?	Sparse table query
min, max	✔	O(1)
gcd, AND, OR	✔	O(1)
sum, xor, product	✘	O(log n) — split into **disjoint** powers of two

For static sums use prefix sums instead.

Where sparse tables shine

- LCA in O(1) via Euler tour + RMQ (Week 22)
- Binary search over a range with O(1) checks: e.g. "longest subarray with gcd > 1"
- Two pointers + O(1) range min/max (CF 689D *Friends and Subsequences*)
- Suffix array LCP queries (Week 29+)

Choosing the structure

	Fenwick	Sparse table	Segment tree
Updates	point (+ range tricks)	none	point (+ lazy)
Query	invertible ops	idempotent ops	any associative
Query time	O(log n)	**O(1)**	O(log n)
Memory	n	n log n	2n–4n
Code	tiny	small	medium

Common pitfalls

- Fenwick with **index 0** → infinite loop (`0 & -0 = 0`)
- Forgetting `r + 1 ≤ n` bound in range-add (allocate n + 2)
- Using a BIT for range **min** with arbitrary updates
- Sparse table: computing `log2` with floating point → off-by-one; precompute `lg[]`
- Sparse table memory: `20 × 10^6` ints = 80 MB — check the limit
- Sparse table for **sum** with overlapping windows → double counting

Practice problems

- CSES 1648 *Dynamic Range Sum Queries* (BIT)
- CSES 1651 *Range Update Queries* (BIT on differences)
- CSES 1647 *Static Range Minimum Queries* (sparse table)
- CSES 1749 *List Removals* (BIT k-th)
- CSES 1739 *Forest Queries II* (2D BIT)
- SPOJ INVCNT *Inversion Count*
- Codeforces 61E *Enemy is weak*
- Codeforces 1354D *Multiset* (BIT k-th, tight memory)
- Codeforces 689D *Friends and Subsequences* (sparse tables)

Summary

- **Fenwick**: `t[i]` covers `(i - lowbit(i), i]`; query strips bits, update adds bits
- Prefix sums with updates, inversions, k-th element, range-add tricks
- **Sparse table**: `sp[k][i]` over windows of length 2^k
- O(1) query for **idempotent** ops via two overlapping windows
- Next week: lazy propagation for general range updates